

Exercícios – Descrição da Sintaxe e da Semântica

Vanessa Beatriz G. Beltrame

24034783-2

1. Sintaxe e semântica

Sintaxe é a forma certa de escrever o código, seguindo as regras da linguagem. Semântica é o que o código quer dizer ou fazer.

Por exemplo:

```
x = 2 + 3;
```

A sintaxe está certa porque está escrito no formato que a linguagem aceita. A semântica é que ele vai fazer a conta e colocar o resultado 5 dentro de `x`.

Então, resumindo, sintaxe é como escrever e semântica é o que o código faz.

2. Sentenças, lexemas e tokens

Uma afirmação errada seria dizer que lexema e token são a mesma coisa.

Eles são parecidos, mas não são iguais. O lexema é o que realmente está escrito no código, como `total` ou `42`. Já o token é a categoria desse pedaço, como identificador ou número.

Por exemplo, `soma`, `x` e `contador` são lexemas diferentes, mas os três podem ser classificados como identificadores.

Uma sentença é uma sequência de tokens que segue as regras da linguagem.

3. Reconhecedores e geradores

Um gerador serve para criar frases que são válidas de acordo com as regras da linguagem.

Já o reconhecedor pega uma frase pronta e verifica se ela está certa ou não.

Por exemplo, usando:

```
<soma> ::= <numero> + <numero>
```

e:

```
<numero> ::= 0 | 1 | 2 | 3
```

A expressão `1 + 2` está certa porque 1 e 2 são números que estão na regra.

Já `1 + x` estaria errada porque `x` não está entre os números permitidos.

Então o reconhecedor basicamente verifica se o que foi escrito segue as regras.

4. BNF e gramáticas livres de contexto

BNF é uma forma de escrever as regras de uma gramática. Ela pode ser usada para mostrar como uma linguagem é formada.

Podemos olhar para ela de duas formas:

Como regras que mostram como montar uma expressão passo a passo.

Como uma forma de mostrar quais frases podem ser aceitas pela linguagem.

A diferença é que uma forma mostra mais como montar e a outra mostra o que pode ser formado.

5. Derivação e árvore sintática

Imagine que um compilador recebe:

$2 + 3 * 4$

Ele precisa entender como essa expressão está organizada.

Podemos começar com:

`<expr>`

Depois:

`<expr> + <expr>`

Depois:

$2 + \text{<expr>} * \text{<expr>}$

E no final:

$2 + 3 * 4$

A árvore sintática ajuda o compilador a entender a ordem da expressão.

Nesse caso, a multiplicação deve ser feita antes da soma. Então primeiro temos $3 * 4$ e depois somamos o 2.

6. Ambiguidade gramatical

Uma gramática é ambígua quando uma mesma frase pode ser entendida de mais de uma maneira.

Por exemplo:

$2 + 3 * 4$

Se a gramática não disser qual operação vem primeiro, podemos pensar em:

$(2 + 3) * 4$

ou:

$2 + (3 * 4)$

Os resultados seriam diferentes: 20 e 14.

Por isso, a gramática precisa deixar claro qual operação tem prioridade.

7. BNF estendida

Uma afirmação errada seria:

"A BNF estendida consegue criar linguagens que a BNF normal não consegue."

Isso está errado porque a EBNF não cria linguagens novas. Ela só deixa as regras mais fáceis de escrever e entender.

Ela possui alguns símbolos que ajudam nisso, como `[ ]` para algo opcional e `{ }` para repetição.

Então, a diferença principal é que a EBNF deixa a escrita das regras mais simples, mas não aumenta o que a BNF consegue representar.

8. Gramáticas de atributos

Uma gramática de atributos é uma gramática que tem algumas informações extras para verificar o código.

Por exemplo:

`x = 5`

Se `x` foi declarado como inteiro, o número 5 também é inteiro. Então está tudo certo.

Podemos pensar assim:

Verificamos o tipo de `x`.

Verificamos o tipo de 5.

Comparamos os dois.

Como os dois são inteiros, não tem problema.

Mas se fosse:

`x = "texto"`

poderia dar erro, porque `x` é inteiro e `"texto"` é texto.

9. Atributos sintetizados e herdados

A diferença está em de onde vem a informação.

O atributo sintetizado recebe informação dos filhos da árvore. A informação vai de baixo para cima.

O atributo herdado recebe informação do pai ou de algum outro nó. Nesse caso, a informação pode ir de cima para baixo.

Então, de forma simples:

Sintetizado: vem dos filhos.

Herdado: vem do pai ou de outro nó.

10. Métodos de semântica dinâmica

A semântica dinâmica serve para explicar o que acontece quando o programa está sendo executado.

Um exemplo seria o comando `while`.

Se temos:

```
while (condição)
```

e a condição for verdadeira, o comando é executado e depois a condição é verificada novamente.

Se a condição for falsa, o `while` termina.

Então podemos acompanhar o programa passo a passo, vendo o que acontece com as variáveis e com os comandos.

Isso é importante porque ajuda a deixar claro como a linguagem deve funcionar quando o programa é executado.